

Problem Set 1-2

Methods

1. Grayscale

The idea is to combine the three channel values (R, G, B) into one channel (0~255, for different levels of black and white) using the mean of the RGB values of each pixel as its new grayscale value. This is implemented by using a double for loop to change the color of each pixel.

2. Binary

The idea is to find a proper threshold (in this problem set, we use the mean of the grayscale image). For cases that we want to emphasize brighter regions, we let every pixel with a value larger than the threshold to be white, and every pixel with a value smaller than the threshold to be black. To emphasize darker regions, we switch the conditions.

However, in this problem set, we adjust the threshold for emphasizing darker regions to be lower, since dark pixels that act as noises in the image are frequent, and we want to filter them out.

3. Color image

The idea of this task is to consider the black regions from the binary image, and paint them to red. For cv2, the code reads color channels in a format of (B, G, R), hence the code for coloring red being `img_output[i][j] = [0, 0, 255]`.

ps1-2_code.py Code printouts

```
import cv2, os
import numpy as np

filename = input("Enter the input image filename: ")
img = cv2.imread(filename)
region = input("Do you want to emphasize brighter regions or darker regions? ")
name, ext = os.path.splitext(filename)

# 1. Display input image
cv2.imshow("input", img)
cv2.imwrite(name + ext, img)

# 2. gray scale
# idea: combine the three channels RGB values to get a single channel grayscale value
image
img_gray = np.zeros((img.shape[0], img.shape[1]), dtype=np.uint8)
for i in range(img.shape[0]):
    for j in range(img.shape[1]):
```

```

        img_gray[i][j] = np.mean(img[i][j])
cv2.imshow("grayscale", img_gray)
cv2.imwrite(name + "_grayscale" + ext, img_gray)

# 3. binary
# idea: use a proper threshold to separate the yellow circuits from the green background
threshold = np.mean(img_gray)
img_binary = np.zeros((img.shape[0], img.shape[1]), dtype=np.uint8)
for i in range(img.shape[0]):
    for j in range(img.shape[1]):
        if region.lower() == "brighter":
            if img_gray[i][j] > threshold:
                img_binary[i][j] = 255
            else:
                img_binary[i][j] = 0
        elif region.lower() == "darker":
            if img_gray[i][j] < threshold - 90: # use a lower threshold because black pixel
noises appear
                # -90 for emphasizing crack2 cracks due to the shades, -15 for emphasizing
crack1 cracks
                # if we use -15, the cracks can be emphasized in a general way
                # to emphasize crack2 cracks and neglect the shades, change -15 to -90
                img_binary[i][j] = 255
            else:
                img_binary[i][j] = 0
cv2.imshow("binary", img_binary)
cv2.imwrite(name + "_binary" + ext, img_binary)

# 4. output color image
# idea: if it is the emphasized area in binary image, paint them red
img_output = img.copy()
for i in range(img.shape[0]):
    for j in range(img.shape[1]):
        if img_binary[i][j] == 255:
            img_output[i][j] = [0, 0, 255] # cv2 reads in BGR
cv2.imshow("output", img_output)
cv2.imwrite(name + "_output" + ext, img_output)

cv2.waitKey(0) # show the images until key is pressed
cv2.destroyAllWindows()

```



